

```

#include "mainwindow.h"
#include <QGridLayout>
#include <QLabel>
#include <QPushButton>
#include <QTextEdit>
#include <QVBoxLayout>
#include <QWidget>

namespace {
const int COLS = 5;
const int ROWS = 5;
const char* kPlayerColors[] = {"#ffb3b3", "#b3d1ff"};

//Helps out use qt to make the UI layout for board set up
QPair<int, int> ringPosition(int i)
{
    int idx = 0;
    for(int c = 0; c < COLS; ++c){
        if(idx++ == i){
            return {0, c};
        }
    }
    for(int r = 1; r < ROWS; ++r){
        if(idx++ == i){
            return {r, COLS - 1};
        }
    }
    for(int c = COLS - 2; c >= 0; --c){
        if(idx++ == i){
            return {ROWS - 1, c};
        }
    }
    for(int r = ROWS - 2; r >= 1; --r){
        if(idx++ == i){
            return {r, 0};
        }
    }
    return {0,0};
}
}

//Refreshes board to when you first start
MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent){
    setWindowTitle("Linked List Monopoly");
    buildUi();
    refresh();
    log_>append("Game started. " + QString::fromStdString(game_.currentPlayer().name) + " goes first.");
}

//Where the buttons get mapped to, as well as the locations of the square plots player can buy
void MainWindow::buildUi(){
    auto* central = new QWidget(this);
    auto* grid = new QGridLayout(central);
    grid->setSpacing(4);

    //One QLabel per node in the linked list
    const auto nodes = game_.board().toVector();
    for (int i = 0; i < static_cast<int>(nodes.size()); ++i) {
        auto* tile = new QLabel(this);
        tile->setAlignment(Qt::AlignCenter);
        tile->setWordWrap(true);
        tile->setMinimumSize(120, 90);
        tile->setFrameStyle(QFrame::Box);
    }
}

```

```

        auto pos = ringPosition(i);
        grid->addWidget(tile, pos.first, pos.second);
        tiles_.push_back(tile);
    }

    // Center panel: status, buttons, log
    auto* center = new QWidget(this);
    auto* v = new QVBoxLayout(center);
    status_ = new QLabel(this);
    status_->setAlignment(Qt::AlignCenter);
    rollBtn_ = new QPushButton("Roll dice and move", this);
    buyBtn_ = new QPushButton("Buy property", this);
    endBtn_ = new QPushButton("End turn", this);
    log_ = new QTextEdit(this);
    log_->setReadOnly(true);

    v->addWidget(status_);
    v->addWidget(rollBtn_);
    v->addWidget(buyBtn_);
    v->addWidget(endBtn_);
    v->addWidget(log_);
    grid->addWidget(center, 1, 1, ROWS - 2, COLS - 2);

    setCentralWidget(central);

    connect(rollBtn_, &QPushButton::clicked, this, &MainWindow::onRoll);
    connect(buyBtn_, &QPushButton::clicked, this, &MainWindow::onBuy);
    connect(endBtn_, &QPushButton::clicked, this, &MainWindow::onEndTurn);
}

void MainWindow::refresh()
{
    const auto nodes = game_.board().toVector();
    const auto& players = game_.players();

    for (int i = 0; i < nodes.size(); ++i) {
        const Property* p = nodes[i];

        QString text = QString::fromStdString(p->name);
        if (p->cost > 0) text += QString("\n$%1").arg(p->cost);
        if (p->owner != -1)
            text += "\nOwner: " + QString::fromStdString(players[p->owner].name);

        // Show a token for every player standing on this node
        QString tokens;
        for (int k = 0; k < static_cast<int>(players.size()); ++k)
            if (players[k].position == p) tokens += QString(" [P%1]").arg(k + 1);
        if (!tokens.isEmpty()) text += "\n" + tokens;

        QString bg = (p->owner == -1) ? "#f4f4f4" : kPlayerColors[p->owner % 2];
        tiles_[i]->setText(text);
        tiles_[i]->setStyleSheet(QString("background:%1; color:black;").arg(bg));
    }

    QString s;
    for (int k = 0; k < static_cast<int>(players.size()); ++k)
        s += QString("P%1: $%2 ").arg(k + 1).arg(players[k].money);
    s += QString("\nCurrent: %1").arg(QString::fromStdString(game_.currentPlayer().name));
    status_->setText(s);

    rollBtn_->setEnabled(!game_.hasRolled());
    buyBtn_->setEnabled(game_.canBuy());
    endBtn_->setEnabled(game_.hasRolled());
}

```

```
void MainWindow::onRoll()
{
    log_>append(QString::fromStdString(game_.roll()));
    refresh();
}

void MainWindow::onBuy()
{
    log_>append(QString::fromStdString(game_.buy()));
    refresh();
}

void MainWindow::onEndTurn()
{
    game_.endTurn();
    refresh();
}
```